



review this report and implementation strategy, then make it cohesive as a rewrite:

ALWAYS ON – Executive Summary & Technical Overview

Executive Summary (Plain-English)

Imagine a single desktop computer that keeps your entire drone-based business running all the time. The system automatically collects data from tiny, inexpensive drones, processes that data into maps and insights, handles customer orders through a friendly online forum, and even runs virtual pilots to fine-tune how those drones fly. All of this happens without you having to manually restart anything – the computer watches for new software updates, pulls fresh images, and keeps every component healthy.

The key points are:

- **One machine, one hub** – a Kubuntu 26 laptop with KDE Plasma manages everything from payments to simulations.
- **Drones on the field** – Raspberry Pi 5 boards equipped with LoRa radios collect data and talk back when they land.
- **Community & commerce** – A Mastodon server is your storefront and knowledge base; customers can ask questions, place orders, or customize their setup via a chat-bot powered by a language model.
- **Automated analysis** – Images from the drones are automatically turned into maps, while simulations refine flight plans before you send a drone out.
- **Always on, low cost** – The whole stack runs in lightweight containers and only exposes the ports that really need to be public.

Technical Stack Summary

A. Platform & Runtime

- **Host OS:** Kubuntu 26 (Linux kernel 6.x), KDE Plasma 6.7 desktop environment.
- **Container Runtime:** Podman, rootless – no daemon, runs as a normal user.
- **Service Orchestration:** systemd units + timers for daily image refresh; optional Watchtower container for continuous monitoring.

B. Core Services & Their Responsibilities

Service	Role	Key Ports (Public)	Internal Dependencies
Corda	Payment ledger, receipt generation, private-user list	8888 RPC, 10000 Web UI (HTTPS via reverse proxy)	PostgreSQL (shared DB), Java runtime
MeshChatX	LoRaWAN gateway + WebSocket server for drones	9000 WSS	LoRa radio driver on host, OpenClaw status endpoint
WebODM	Geospatial image processing pipeline	8000 REST & WebUI (HTTPS)	Redis queue, PostgreSQL (optional), drone-image folder
LM Studio	Language model inference server	5000 FastAPI	GPU/CPU resources, Python runtime
OpenClaw	AI agent that posts to Mastodon	8001 status (internal)	Calls LM Studio API, uses Mastodon REST API
Gazebo + ArduPilot + ROS + SB3	Full vehicle & robot arm simulation	11345 ROS Master, 5760 MAVLink UDP, 5000 SB3 API (internal)	ROS topics, ArduPilot telemetry, world files

C. Data Flow and Inter-Service Communication

- **Drone → MeshChatX:** LoRaWAN packets are received by the host's radio hat; MeshChatX forwards messages via WebSocket to OpenClaw.
- **OpenClaw ↔ LM Studio:** Text prompts and responses over HTTP (FastAPI).
- **Drone → Local Folder → WebODM:** Upon landing, drones push JPEG/RAW images to a shared directory; WebODM picks them up from the folder and processes them into orthomosaics.
- **Simulation Results:** Gazebo writes flight logs and arm trajectories to `/results`; these files are then available for analysis or export to external systems.

D. Security & Exposure

- Only essential ports (8000, 8888, 9000, 11345, 5760) are mapped to the host; all other traffic stays within Podman's internal network.
- TLS termination via Traefik/NGINX on the host; Mastodon interactions use HTTPS and OAuth tokens stored in `~/.config/mastodon`.
- Each pod runs as a dedicated non-root user with read-only mounts for code, ensuring least privilege.

E. Why This Stack is Valuable

1. **Low-Cost Autonomy** – A single desktop computer can orchestrate multiple drones and a complex vehicle simulation without expensive servers or cloud services.
2. **Full Data Loop** – From field data collection (LoRaWAN + Wi-Fi imagery) to on-device processing, AI inference, public communication, and simulation refinement—all automated.

3. **Scalability & Modularity** – Adding a new drone is as simple as attaching another LoRa radio hat; the same Podman pods can handle multiple vehicles because they communicate over the internal network.
4. **Cross-Platform Compatibility** – The Raspberry Pi 5 units run KDE Plasma 6.7 (KALIOS) and are fully compatible with the host's desktop environment, simplifying UI sharing or remote management.
5. **Regulatory & Compliance Ready** – Corda provides a tamper-proof ledger; all data paths are logged by systemd and can be audited. The system can be extended to support GDPR/CCPA compliance via proper data retention policies.

F. Future Enhancements

- Automatic scaling of LM Studio workers based on request load.
- Container image signing for secure updates.
- Edge-to-cloud sync for long-term storage or analytics dashboards.
- Integration with additional IoT protocols (e.g., NB-IoT, Sigfox) if required.

ALWAYS ON – Executive Summary & Technical Overview

1. Installation Script (Run as root)

NOTE: Execute this block *once* on your Kubuntu 26 machine to provision users, networks, systemd units and the daily update timer.

```
#!/usr/bin/env bash
set -euo pipefail

# ----- 0 Environment -----
sudo apt update && sudo apt upgrade -y
sudo apt install podman podman-docker systemd-user-container \
    redis-server postgresql lora-tools

# ----- 1 Users & Network -----
for svc in meshchatx webodm lmstudio openclaw mastodon_node ardupilot gazebo; do
    sudo adduser --disabled-password --gecos "" $svc
done

podman network create sim-net

mkdir -p /etc/pod_run_commands
chmod 700 /etc/pod_run_commands

# ----- 2 PostgreSQL -----
sudo -u postgres psql <<'EOSQL'
CREATE DATABASE app_shared;
CREATE USER app_user WITH PASSWORD 'StrongPass!';
GRANT ALL PRIVILEGES ON DATABASE app_shared TO app_user;
```

EOSQL

```
# ----- 3␣ Helper Script -----
cat > /usr/local/bin/run_pod.sh <<'EOF'
#!/usr/bin/env bash
set -euo pipefail
svc=$1
cmd=$(sed -n 1p "/etc/pod_run_commands/${svc}")
exec $cmd
EOF
chmod +x /usr/local/bin/run_pod.sh

# ----- 4␣ Systemd Units -----
cat > /etc/systemd/user/corda-node.service <<'EOF'
[Unit]
Description=Corda Node
After=network.target postgresql.service

[Service]
Type=simple
User=corda_node
ExecStart=/usr/local/bin/run_pod.sh corda-node
Restart=on-failure
StandardOutput=syslog
StandardError=syslog
SyslogIdentifier=corda-node

[Install]
WantedBy=default.target
EOF

# (Repeat similar blocks for meshchatx-server, webodm-web,
# lmstudio-headless, openclaw-agent, ros-master, gazebo-sim)
# ... omitted here for brevity - see full report below.

# ----- 5␣ Daily Update Timer -----
cat > /usr/local/bin/pod_update_all.sh <<'EOF'
#!/usr/bin/env bash
set -euo pipefail
for svc in $(ls /etc/pod_run_commands); do
    echo "Updating $svc"
    podman pull "$(sed -n 1p "/etc/pod_run_commands/${svc}")" 2>/dev/null || true
    systemctl --user stop "${svc}.service" 2>/dev/null || true
    systemctl --user start "${svc}.service"
done
EOF
chmod +x /usr/local/bin/pod_update_all.sh

cat > /etc/systemd/user/pod-update.timer <<'EOF'
[Unit]
Description=Daily Pod Image Update

[Timer]
OnCalendar=daily
Persistent=true
```

```
[Install]
WantedBy=timers.target
EOF
```

```
cat > /etc/systemd/user/pod-update.service <<'EOF'
```

```
[Unit]
Description=Run pod_update_all.sh
```

```
[Service]
Type=oneshot
ExecStart=/usr/local/bin/pod_update_all.sh
StandardOutput=syslog
StandardError=syslog
SyslogIdentifier=pod-update
```

```
[Install]
WantedBy=default.target
EOF
```

```
# ----- 6. Enable Units -----
```

```
systemctl --user daemon-reload
```

```
for svc in corda-node meshchatx-server webodm-web lmstudio-headless openclaw-agent ro
    systemctl --user enable --now ${svc}.service
```

```
done
```

```
systemctl --user enable --now pod-update.timer
```

```
echo "Installation complete. Reboot or log out/in to activate all units."
```

After running the script

- All services are now defined as user-level systemd units.
- The daily image refresh timer is active.
- The pods will start automatically on boot.

2. Architecture Overview

(Diagram omitted - refer to the text description in the previous reply.)

1. Host: Kubuntu 26, KDE Plasma 6.7.

2. Pods:

- * corda-node - payment ledger (PostgreSQL).
- * meshchatx-server - LoRaWAN gateway + WebSocket bridge.
- * webodm-web - image ingestion & processing.
- * lmstudio-headless - language-model inference.
- * openclaw-agent - Mastodon bot (posts to forum).
- * ros-master, gazebo-sim - simulation stack (ArduPilot, ROS, Gazebo, SB3).

All pods share the internal sim-net network; only public ports are exposed via Traefik.

3. Detailed Development Plan

PhaseDurationKey Tasks

0 - Preparation1 dayInstall base packages, create users, set up sim-net.

1 - Core Services2 daysWrite Podman run commands (/etc/pod_run_commands/*), systemd u

2 - Database & Ledger0.5 daySpin up PostgreSQL, configure Corda node.

3 - LoRaWAN Gateway1 dayConfigure radio hat, test MeshChatX reception.

4 - Image Pipeline1 dayDeploy WebODM, set up image ingestion folder, verify processin

5 - AI Stack2 daysBuild LM Studio image, deploy OpenClaw agent, integrate Mastodon.

6 - Simulation3 daysInstall ROS2, Gazebo, ArduPilot; create world file; validate tele

7 - Automation & Monitoring1 dayEnable daily update timer, optional Watchtower contain

8 - Testing & Validation2 daysUnit tests, integration flow, performance benchmarks, s
9 - Documentation1 dayREADME, deployment guide, configuration reference, runbook, bac
4. Acceptance Criteria

1. All services start on boot and stay healthy for ≥ 72 h without manual restarts.
2. A drone can:
 - * send a LoRa packet $\rightarrow$ MeshChatX receives it;
 - * upload an image via Wi-Fi $\rightarrow$ WebODM processes it;
 - * trigger an OpenClaw post $\rightarrow$ Mastodon displays the content;
 - * receive payment confirmation in Corda within 5 min.
3. Simulation pod completes at least one mission cycle and writes JSON logs to /resul
4. Daily image refresh timer pulls newer images and restarts pods without downtime.
5. Security audit passes: only ports listed in the architecture are open, all contain
5. Next Steps for Agentic AI

1. Verify that /usr/local/bin/run_pod.sh correctly executes the stored run commands.
 2. Populate the omitted systemd unit files (meshchatx-server, webodm-web, etc.) with
 3. Deploy Traefik as a pod, configure dynamic providers to expose public ports 8000,
 4. Configure LoRaWAN radio hat on the host; test packet reception with a simple scrip
 5. Test end-to-end flow: simulate a drone by sending a LoRa packet (or using a PC LoR
 6. Run automated tests (pytest for LM Studio, JUnit for Corda) inside temporary conta
 7. Document all steps in a README.md, including troubleshooting tips (e.g., checking
- Final Note

This file now contains the full installation script at the top followed by a chronolo
Feel free to copy-paste it into /home/scottw/ALWAYS_ON.md or any other location you p

9 - Automation & Updates (cont.)

Task	Detail
*****	*****
Daily Image Refresh	Systemd timer `pod-update.timer` executes `/usr/local/bin
Watchtower (Optional)	Run `podman run -d --name watchtower \
	-v /var/run/docker.sock:/var/run/docker.sock \
	containrrr/watchtower --interval 3600` for real-time monitoring of image updates

10 - Testing & Validation

Test	Purpose	Method
*****	*****	*****
Unit Tests	Verify individual service logic (e.g., Corda RPC, LM Studio infer	
Integration Tests	End-to-end flow: Drone $\rightarrow$ MeshChatX $\rightarrow$ OpenClaw $\rightarrow$ LM Studio -	
Performance Benchmarks	Ensure 24/7 uptime under load (e.g., 10 concurrent dr	
Failover Tests	Simulate pod crash and verify auto-restart. Kill a container	
Security Audits	Validate TLS, firewall rules, and least-privilege mounts. I	

11 - Documentation & Hand-Off

1. **README.md** - high-level architecture, prerequisites, and build steps.
2. **Deployment Guide** - step-by-step instructions to set up the host, create users
3. **Configuration Reference** - list of all environment variables per service with c
4. **Operational Runbook** - troubleshooting common issues: service not starting, cor
5. **Backup & Recovery** - scripts to snapshot PostgreSQL data, export simulation res
6. **Change Log** - track versioning of images, configuration changes, and deployment

12 - Risk Assessment & Mitigation

Risk	Impact	Likelihood	Mitigation
Container OOM (e.g., LM Studio GPU memory)	Service crash	Medium	Set `--memory`
LoRa packet loss	Incomplete telemetry	Low	Implement redundancy: store packets
Unauthorized access to Mastodon API	Data breach	Low	Store OAuth tokens in env
Disk space exhaustion (image ingestion)	System halt	Medium	Auto-rotate logs,
Dependency breakage (e.g., image pull failure)	Deployment freeze	Low	Pin image

13 - Timeline & Milestones

Week	Deliverable
0	Environment set up, Podman installed.
1	Core network and user accounts created.
2	Systemd unit templates finalized; first pod (Corda) deployed.
3	MeshChatX + LoRa integration complete.
4	WebODM image ingestion pipeline operational.
5	LM Studio & OpenClaw AI stack functional.
6	Gazebo/ArduPilot simulation running and exporting results.
7	Automation scripts (`pod_update_all.sh`, timers) in place.
8	Full end-to-end testing, performance benchmarks, security audit.
9	Documentation hand-off, operational runbook finalized.

14 - Acceptance Criteria

- All services start automatically on boot and remain healthy for 72 hours without manual intervention.
- A drone can transmit a LoRa packet, upload an image via Wi-Fi, have it processed by the system.
- The simulation pod runs at least one full mission cycle, outputs JSON logs to `/recovery`.
- Systemd timers successfully pull newer images and restart pods without downtime during updates.
- Security audit passes with no exposed ports beyond the defined public list.

15 - Next Steps for the Agentic AI

1. **Implement Scripts** - Create `/usr/local/bin/pod\_update\_all.sh` that iterates over all pods and updates them.
2. **Write Systemd Units** - For each service, generate a unit file that references the scripts.
3. **Configure Traefik** - Deploy as a pod with dynamic config files mapping public IP to internal services.
4. **Deploy LoRa Stack** - Install `lora-tools`, configure the radio hat, and validate communication.
5. **Set Up WebODM** - Pull `webodm:latest`, mount `/home/scottw/webodm\_output`, and expose port 8080.
6. **Deploy AI Stack** - Build LM Studio image (`python:3.11-slim` + PyTorch), expose port 8080.
7. **Simulation Setup** - Install ROS2 Foxy, Gazebo 11, SB3; create a world file, launch simulation.
8. **Automation & Monitoring** - Configure `pod-update.timer`, enable Watchtower if containerd is used.
9. **Testing** - Run unit, integration, performance, failover tests; document results.
10. **Documentation** - Populate README, deployment guide, configuration reference, and user manual.

End of Development Plan

The concept is strong: an on-premises, containerized operations hub that closes the loop between real-world drone data and AI-driven simulation.

Below is a cohesive rewrite that treats the proposal as an implementation strategy rather than a conceptual overview.

ALWAYS ON

On-Premises Drone Operations, Mapping, Simulation, and Customer Platform

Executive Summary

****ALWAYS ON**** is an on-premises operations platform for a drone-focused business. It
Raspberry Pi-based field nodes collect telemetry through LoRa or LoRaWAN-compatible
The design goal is not “one container per idea.” It is a reliable, auditable platform

- Public-facing applications are limited to a reverse proxy and explicitly approved v
- Telemetry, databases, simulation middleware, inference endpoints, and control inter
- Each workload has a defined owner, storage location, backup method, health check, i
- Automation improves uptime without allowing untested upstream releases to disrupt

The initial deployment target is a Kubuntu workstation with Podman running rootless c

Scope and Operating Principles

Primary capabilities

- Receive low-bandwidth drone telemetry through a host-connected LoRa radio or a ded
- Receive mission imagery over authenticated Wi-Fi or a controlled upload endpoint.
- Produce photogrammetry outputs through WebODM or an ODM-compatible processing serv
- Run local LLM inference through LM Studio Core in headless mode.
- Support a narrowly scoped AI agent for internal summaries, alerts, support drafts,
- Run Gazebo, ArduPilot, ROS 2, and Stable-Baselines3 workloads for non-production s
- Maintain operational records, audit logs, backup snapshots, and configuration histo

Non-goals for the first release

- Autonomous customer payments or payment settlement through an experimental ledger s
- Direct internet exposure of ROS, MAVLink, LoRa gateway internals, PostgreSQL, LM S
- Unattended “latest tag” updates for core operational services.
- Treating Mastodon as an order-management platform or authoritative customer databa
- Running flight-critical control loops through a public internet service.

Architecture

Host platform

Component	Initial standard
Host OS	Supported Kubuntu LTS release, kept on a defined patch cadence
Container engine	Rootless Podman
Service management	systemd with Quadlet container, network, volume, and pod defini
External ingress	Traefik, Caddy, or NGINX reverse proxy
Internal DNS/service discovery	Podman network aliases and explicit environment co
Persistent storage	Dedicated data volumes or mounted filesystem paths with backup
Observability	systemd journal, Podman logs, health checks, metrics, and disk moni
Backup target	Encrypted local external storage and/or approved off-site repositor

Podman Quadlet is preferred because it makes containers first-class systemd-managed s

Network zones

Zone	Purpose	Internet exposure
Edge network	Reverse proxy and selected web applications	Yes, only through TLS
Application network	WebODM, AI agent, internal APIs, application services	No d

Data network	PostgreSQL, Redis, object/file storage	No direct public access
Telemetry network	LoRa/LoRaWAN ingestion and message routing	No direct public access
Simulation network	ROS 2, Gazebo, MAVLink, ArduPilot, training workloads	No direct public access
Management network	SSH, dashboards, host administration	VPN or allowlisted access

A container should join only the networks it requires. For example, the reverse proxy

Core service roles

Service area	Preferred role	Exposure	Notes
Reverse proxy	TLS termination, routing, access logs, rate limiting	Public 80/443	
Customer/community frontend	Mastodon or separate web storefront	Public through proxy	
Operational database	PostgreSQL	Internal only	Use separate databases and roles
Cache/queue	Redis or supported queue backend	Internal only	Never expose Redis directly
Mapping	WebODM plus supported worker/node components	Through proxy or VPN	WebODM is public
Telemetry gateway	LoRa/LoRaWAN bridge and message normalizer	Internal only	VPN or allowlisted access
AI inference	LM Studio Core or equivalent local inference server	Internal only	
AI agent	OpenClaw or custom service	Internal only	Use approval gates for external access
Simulation	ArduPilot, Gazebo, ROS 2, SB3	Internal only	Separate development/testing network
Ledger/records	Conventional order/payment system initially; optional Corda evaluation		

Data Flows

Telemetry path

1. A field node records position, health, mission state, sensor data, and event messages.
2. The node transmits compact telemetry through LoRa or LoRaWAN.
3. A dedicated gateway service validates device identity, timestamps packets, detects duplicates, and normalizes the data.
4. A normalization service publishes approved telemetry to internal consumers such as the database and simulation.
5. The original packet and normalized event are retained according to the data-retention policy.

The implementation must decide whether the radio system is raw LoRa, LoRaWAN, Meshtastic, or something else.

Imagery and mapping path

1. The drone lands or enters a trusted Wi-Fi zone.
2. A transfer client uploads imagery and mission metadata to a per-mission staging area.
3. A validation process checks file integrity, available disk space, metadata completeness, and other criteria.
4. An ingestion service creates a WebODM project and task through its supported interface.
5. WebODM processes imagery and produces deliverables such as orthomosaics, point clouds, and maps.
6. Completed deliverables move to a read-only results location, with status returned to the client.

Do not assume that simply placing JPEG or RAW files in a shared directory automatically makes them available.

AI and communications path

1. Internal services generate structured events, such as “mapping job completed,” “telemetry received,” or “simulation started.”
2. The AI agent receives only the minimum necessary event data.
3. It calls the local LM Studio service for a draft, classification, summary, or response.
4. A policy layer determines whether the action is internal-only, requires operator approval, or is public.
5. Mastodon publishing uses scoped OAuth credentials stored in a secrets manager or provided by the user.

LM Studio’s current tooling supports headless operation and service control through its API.

Simulation path

1. A mission configuration is copied into a versioned simulation workspace.
2. ArduPilot SITL, Gazebo, ROS 2 nodes, and any SB3 training/evaluation services start.
3. Simulation outputs include mission logs, trajectory files, vehicle state, test parameters, and other data.
4. Results are written to a host-mounted, versioned output directory such as ``/srv/ai``.
5. Only approved mission parameters progress from simulation to field-test preparation.

ROS discovery, MAVLink, and simulation ports must remain private. They are development

Security and Reliability Model

Exposure policy

Public exposure is limited to:

- TCP 80, redirected to HTTPS where appropriate.
- TCP 443 for approved web routes.
- VPN access for administration, remote dashboards, or trusted operator access.

The following remain internal-only:

- PostgreSQL.
- Redis.
- LM Studio API.
- Corda RPC or administrative interfaces.
- LoRa gateway administrative interfaces.
- ROS 2 middleware.
- MAVLink UDP endpoints.
- Gazebo transport.
- WebODM worker and NodeODM interfaces.
- Podman service socket.
- Host metrics endpoints, unless protected through VPN and authentication.

Identity and secrets

- Create a separate Linux service account only where it provides meaningful filesystem permissions.
- Prefer rootless Podman containers owned by a dedicated deployment user, rather than root.
- Use root-owned system services for host-level requirements such as radio drivers, network interfaces, etc.
- Use ``systemd`` credentials, a secrets manager, or protected environment files for API keys and passwords.
- Do not embed passwords such as ``StrongPass!`` in deployment scripts, source files, or configuration files.
- Use separate PostgreSQL roles and databases per application instead of one broadly-permissive role.

Update policy

Automatic updates are appropriate only for low-risk, well-tested services. For operational services, updates require testing and approval.

Podman provides native auto-update support for labeled, ``systemd``-managed services, including:

Workload class	Update policy
Reverse proxy and monitoring	Automatic updates may be acceptable after validation
AI agent and noncritical web services	Scheduled updates with rollback plan
WebODM, databases, and queues	Staged manual updates, backup first
Simulation stack	Version-pinned environments to preserve repeatability
Flight-adjacent telemetry/control services	Manual, tested release process only
Ledger/payment services	Formal change control, backup, migration test, and rollback

Do not use Watchtower for this Podman-centric deployment. It is primarily designed as

Health and recovery

Each service must have:

- A health endpoint or command.
- Restart policy and restart-rate limit.
- Resource limits for CPU, memory, and open files.
- A persistent volume declaration where state is required.
- A backup classification.
- A dependency declaration only when startup ordering is genuinely necessary.
- A documented rollback procedure.

A service being “running” is not equivalent to being healthy. The acceptance test sho

Implementation Strategy

Phase 0 – Design validation

****Duration:**** 2-5 days

- Confirm the host hardware, GPU support, RAM, NVMe capacity, and backup capacity.
- Choose one supported Kubuntu release and record exact package versions.
- Define the radio protocol and hardware architecture.
- Decide whether Mastodon is external, self-hosted, or replaced with a conventional s
- Decide whether Corda is a real business requirement or an exploratory component.
- Create a threat model covering public users, field nodes, operators, lost drones, c
- Define data classes: telemetry, imagery, customer information, credentials, payment

****Exit criteria:**** Architecture decisions are recorded, domains and trust boundaries

Phase 1 – Host foundation

****Duration:**** 1-2 days

- Install Podman, Quadlet support, firewall tooling, storage monitoring, and required
- Configure automatic security updates for the host separately from container updates
- Establish a dedicated deployment account with lingering enabled if rootless user se
- Create `/srv/always-on/` storage hierarchy for configuration, incoming imagery, pro
- Configure firewall rules with default-deny inbound policy.
- Set up encrypted backup storage and perform one restore test.

****Exit criteria:**** The host reboots cleanly, the deployment account’s services start

Phase 2 – Networking, ingress, and observability

****Duration:**** 1-2 days

- Define Quadlet-managed internal networks.
- Deploy the reverse proxy with TLS, access logs, rate limits, and authentication whe
- Expose only approved HTTPS routes.
- Add baseline monitoring for disk use, memory pressure, container restarts, failed s
- Configure centralized logs or a structured journal retention policy.

****Exit criteria:**** External scanning confirms that only the approved ingress ports are

Phase 3 – Data and mapping services

****Duration:**** 2–4 days

- Deploy PostgreSQL with separate databases, roles, persistent storage, scheduled back
- Deploy Redis only if required by the selected application stack.
- Deploy WebODM using its supported component model.
- Build a controlled imagery-ingestion service that creates processing tasks and reco
- Establish mission directory conventions and output retention rules.

WebODM's official materials describe it as a drone-image processing platform that can

****Exit criteria:**** A known image set is uploaded, processed, archived, and retrievabl

Phase 4 – Telemetry gateway

****Duration:**** 2–4 days

- Install and validate the radio hardware and host driver.
- Implement gateway authentication, device registration, packet logging, and duplica
- Persist raw packets before downstream processing.
- Build a normalized telemetry schema and test message delivery.
- Simulate loss, duplicate packets, malformed packets, and delayed delivery.

****Exit criteria:**** A field node can send telemetry that is received, authenticated, s

Phase 5 – AI and communications

****Duration:**** 2–3 days

- Install LM Studio Core or headless LM Studio service.
- Select and version-pin the model(s) used for each function.
- Set GPU/CPU/RAM limits and concurrency limits.
- Deploy the AI agent with narrowly scoped permissions.
- Require explicit approval for public posts, financial statements, pricing, legal co
- Store OAuth credentials securely and rotate them on a defined schedule.

****Exit criteria:**** The system can generate internal mission summaries and customer-re

Phase 6 – Simulation and autonomy development

****Duration:**** 3–5 days

- Deploy version-pinned ROS 2, Gazebo, ArduPilot SITL, and SB3 environments.
- Use a reproducible scenario definition for each mission test.
- Store simulation parameters, environment versions, and results with every run.
- Validate MAVLink telemetry only on the simulation network.
- Define promotion rules from simulation to bench testing and then field testing.

****Exit criteria:**** A representative mission executes in simulation, exports structur

Phase 7 – Automation and operational hardening

****Duration:**** 2–3 days

- Add health checks and restart behavior for all service units.
- Enable Podman auto-update only for approved, low-risk workloads.
- Create a staging or test profile for image upgrades of critical services.
- Set database backup, retention, and restore drills.
- Add disk quotas or threshold alerts for imagery and simulation output.
- Document emergency shutdown, credential revocation, restore, and rollback procedures.

****Exit criteria:**** Failure injection confirms controlled recovery from a container crash.

Deployment Pattern

Recommended service definition approach

Use one Quadlet definition per container or pod, plus explicit network and volume definitions.

A typical service should declare:

- Fully qualified and pinned image reference.
- Container name.
- Network membership.
- Volume mounts.
- Read-only root filesystem where practical.
- User and group mapping.
- Environment files or systemd credentials.
- Health check.
- CPU and memory limits.
- Explicit port exposure only when necessary.
- Auto-update label only if approved.

Quadlet is intended to generate systemd services from declarative container-unit files.

Example operational layout

```
```text
/srv/always-on/
├─ config/
│ ├─ reverse-proxy/
│ ├─ telemetry/
│ ├─ webodm/
│ ├─ ai-agent/
│ └─ simulation/
├─ data/
│ ├─ postgres/
│ ├─ redis/
│ ├─ incoming-images/
│ ├─ webodm-media/
│ └─ telemetry/
├─ results/
│ ├─ mapping/
│ └─ simulation/
├─ backups/
│ ├─ postgres/
│ ├─ configurations/
│ └─ manifests/
└─ logs/
```

Use stable host paths or named Podman volumes for persistent state. Never depend on a container's writable layer for database data, mission imagery, model files, or simulation output.

## Acceptance Criteria

The platform is accepted only when all of the following are demonstrated and recorded.

### Reliability

- Core services start automatically after a host reboot.
- Core services run for at least 72 hours under representative workload without manual restarts.
- A controlled container failure triggers expected recovery without corrupting persistent data.
- Alerting identifies service failures, low disk space, backup failures, and repeated restart loops.

### Drone operations

- A registered field node transmits telemetry that is authenticated, timestamped, stored, and visible to authorized operators.
- A mission image set uploads through the approved path and produces a tracked WebODM processing task.
- Mapping outputs are stored in the assigned mission results location with traceable source metadata.
- Simulation completes at least one representative mission and writes structured logs to the results directory.

### AI and communications

- LM Studio responds only on the internal application network.
- The AI agent can generate internal summaries and audited draft messages.
- External Mastodon posts require approval unless a specific, low-risk automated-post rule has been documented and tested.
- OAuth tokens and database credentials never appear in logs, unit files, shell history, or source control.

### Security

- External scanning finds only the approved ingress ports.
- TLS is enforced for public routes.
- Databases, Redis, ROS, MAVLink, LM Studio, LoRa administration, and container-engine interfaces are not externally reachable.
- Containers use least-privilege user mappings, minimal mounts, and resource limits.

- A backup restoration drill successfully restores a database and selected mission results.

### Key Risks and Controls

Risk	Impact	Control
“Latest” image update breaks a core service	Operational outage or data incompatibility	Pin versions; test updates; use auto-update only for low-risk services
LLM resource contention	Mapping, telemetry, or host responsiveness degraded	Set memory/CPU/GPU limits, queue requests, and isolate inference workloads
Disk exhaustion from imagery or logs	Processing failure and host instability	Capacity alerts, retention policy, quotas, and backup/export workflow
Public exposure of control interfaces	Compromise of simulation, data, or potentially field operations	Reverse proxy only; private networks; VPN/allowlist administration
Radio packet loss or replay	Incomplete or misleading telemetry	Message IDs, timestamps, acknowledgments, deduplication, local edge buffering
Shared database credentials	Cross-service compromise	Separate DB roles, secrets management, least privilege
AI agent publishes incorrect content	Reputational or contractual harm	Approval gates, content policy, audit logs, constrained tool permissions
Payment/ledger overreach	Legal, financial, and compliance risk	Use established payment providers initially; keep Corda as a separately evaluated project

### Delivery Milestones

Milestone	Deliverable
M0	Architecture decision record, threat model, hardware and storage plan
M1	Hardened host, Podman/Quadlet base, firewall, backup restore test
M2	Reverse proxy, private networks, logging, monitoring, and alerts
M3	PostgreSQL, imagery ingestion, and validated WebODM processing
M4	Authenticated LoRa/LoRaWAN telemetry ingestion and storage
M5	Internal LM Studio service and policy-controlled AI agent
M6	Reproducible ArduPilot/Gazebo/ROS 2 simulation workflow
M7	Recovery testing, update policy, security review, and runbook
M8	End-to-end acceptance demonstration and operational handoff

### Immediate Next Actions

1. Replace the proposed installation script with declarative Quadlet files and an environment-specific configuration repository.
2. Choose the exact telemetry protocol and radio hardware before selecting or writing the gateway service.

3. Separate customer communications from payment processing: keep Mastodon as a community channel, and use a conventional order/payment workflow unless there is a documented reason to deploy Corda.
4. Deploy PostgreSQL, WebODM, and the imagery-ingestion adapter first; this establishes the highest-value field-data loop.
5. Run LM Studio as an internal headless service and place the AI agent behind policy controls, approval gates, and audit logging.<sup>[1]</sup> <sup>[2]</sup>
6. Keep ROS, Gazebo, MAVLink, and all simulation interfaces entirely internal.
7. Implement backups and restoration testing before enabling automated updates or accepting operational data.
8. Add automated update capability only after a staging path, pinned releases, health checks, rollback procedures, and service ownership are in place. Podman's native update workflow is the appropriate baseline for systemd-managed Podman services.<sup>[3]</sup>

\*\*

1. <https://lmstudio.ai/docs/developer/core/headless>
2. <https://lmstudio.ai/blog/0.4.0>
3. <https://docs.podman.io/en/v5.0.1/markdown/podman-auto-update.1.html>
4. <https://hub.docker.com/r/opendronemap/nodeodm>
5. <https://github.com/podman-container-tools/podman/discussions/20218>
6. <https://cran.isid.ac.in/web/packages/rlmstudio/vignettes/headless-config.html>
7. [https://www.reddit.com/r/podman/comments/1n2tj1k/updating\\_an\\_containers\\_image\\_within\\_a\\_systemd\\_pod/](https://www.reddit.com/r/podman/comments/1n2tj1k/updating_an_containers_image_within_a_systemd_pod/)
8. <https://webodm.org/download/>
9. <https://docs.opendronemap.org/tutorials/>
10. <https://github.com/lmstudio-ai/lmstudio-bug-tracker/issues/187>
11. <https://major.io/p/podman-quadlet-automatic-updates/>